

■ Hikmah AI – Team Collaboration Guide (Docker + GitHub)

This document provides an end-to-end guide for how the Hikmah AI team collaborates efficiently using Docker and GitHub. It ensures that every team member can work in a consistent environment, contribute effectively, and deploy code seamlessly.

■ 1. Team Collaboration Overview

Hikmah AI is a collaborative AI development project involving multiple components such as a FastAPI backend, React/Next.js frontend, LangChain AI core, and automated data pipelines. Docker standardizes development environments, while GitHub manages source control and team contributions.

■ 2. Recommended Folder Structure

```
hikmah-ai/ ■ ■■■■ backend/ # FastAPI backend ■ ■■■■ app/ ■ ■ ■■■■ main.py ■ ■ ■■■■
routes/ ■ ■■■■ requirements.txt ■ ■■■■ Dockerfile ■ ■■■■ frontend/ # React/Next.js
frontend ■ ■■■■ src/ ■ ■■■■ package.json ■ ■■■■ Dockerfile ■ ■■■■ data_pipeline/ #
Automated data scraping and embeddings ■ ■■■■ pipeline_runner.py ■ ■■■■ Dockerfile ■
■■■■ ai_core/ # LangChain, Agents, and RAG logic ■ ■■■■ __init__.py ■ ■■■■
vector_store.py ■ ■■■■ docker-compose.yml # Runs all services together ■■■■ .env #
Environment variables ■■■■ docs/ # Team documentation
```

■ 3. Using Docker for Development

Docker provides isolated containers for the backend, frontend, and databases to ensure consistency across all team members. Each developer can build and run the project locally using Docker without worrying about dependency conflicts.

■ *Common Docker Commands*

- `docker build -t hikmah-backend . # Build Docker image`
- `docker run -d -p 8000:8000 hikmah-backend # Run container`
- `docker ps # List running containers`
- `docker stop # Stop a container`
- `docker logs # View logs`
- `docker-compose up --build # Run multiple services`
- `docker-compose down # Stop all containers`

■ 4. Using GitHub for Collaboration

GitHub is used for version control and team collaboration. Each developer works on their own branch, commits changes, and submits pull requests for review before merging into the main branch.

■ *Common Git Commands*

- `git clone https://github.com/Hammad-Sahb/Hikmah_AI.git # Clone repo`
- `git checkout -b abdullah-dev # Create new branch`
- `git add . # Stage changes`
- `git commit -m 'Added Docker setup' # Commit changes`
- `git push origin abdullah-dev # Push branch`
- `git pull origin main # Pull latest updates`
- `git merge main # Merge latest into branch`

■ 5. Collaboration Workflow

- Each member clones the repository and creates a new branch for their feature or task.
- All development and testing occur inside Docker containers for consistency.
- Before pushing, test your code inside the container to ensure no errors.
- Push changes to your branch and create a Pull Request (PR) for review.
- Once approved, the branch is merged into the main branch by the project lead.
- Periodically, pull the latest changes from the main branch to stay updated.

■ 6. Best Practices

- Always use Docker for local testing before pushing changes.
- Keep your requirements.txt and package.json updated.
- Use meaningful commit messages (e.g., 'Added Opportunity Pipeline').
- Store sensitive data in the .env file and never push it to GitHub.
- Regularly pull updates from the main branch to avoid merge conflicts.
- Document major changes in the /docs folder.

■ 7. Conclusion

By combining Docker and GitHub, the Hikmah AI team can collaborate effectively, maintain consistent development environments, and ensure smooth deployment across all systems. Docker ensures reproducibility, while GitHub provides transparency and structured version control for all project members.